

Kaspar

A turn-key CME futures trading system that is also the order book simulator you backtest it in.

VENUE
CME Globex

MARKET DATA
MDP3 SBE

ORDER ENTRY
iLink 3

BOOK
MBP & MBO L3

LICENCE
MIT, open source

Kaspar is two things sharing one codebase. It is a production trading system — multicast market data in, full order books reconstructed order by order, an execution algorithm on top, and authenticated iLink 3 sessions out to the exchange. It is also a market simulator: the same books rebuilt from recorded packet captures, with your orders sitting in the price-time queue and filling only when the market actually trades through them.

The same strategy code, the same execution algorithm and the same book run in PCAP replay, in live paper trading, and against the live exchange. You move between them by changing configuration, not by porting anything — so a backtest exercises the code path that will trade.

WHAT IS IN THE BOX

No vendor feed handler, no broker API, no assembly required.

MARKET DATA

Full **MDP3 SBE** decoder: incremental book updates, trades, order-by-order MBO, instrument definitions, snapshot recovery. Sequence gaps are detected and repaired automatically.

ORDER BOOK

Order-by-order reconstruction holding **every resting order individually**, not aggregated depth — which is what makes a queue position, and therefore a fill inference, definable at all.

ORDER ENTRY

iLink 3 session handler: SBE encoding, HMAC authentication, sequence management, negotiation and establishment, primary/secondary failover.

EXECUTION

Shadow-POV, a passive/aggressive percentage-of-volume algorithm that places by following orders in the market. Measured over a full calendar year — see below.

SIMULATION

Three modes off one binary: **PCAP replay**, live multicast paper trading, live iLink execution. Deterministic — a fixed session replays to identical fills.

CONCURRENCY

A custom **C++20 actor framework** — a proven concurrency model, and the same code whether the hot path runs across threads or entirely on one. Scale out by giving actors their own threads; collapse to a single-threaded hot path by grouping them, with no rewrite either way.

STRATEGY AUTHORIZING IN C++ OR RUST

C++ in-process actors for the lowest latency, or Rust across an in-process FFI interop — no socket, no serialisation between the strategy and the book.

MEASUREMENT

An execution-cost harness that runs the algorithm as a two-sided maker over recorded sessions, one row per window, plus a parameter sweep over rate, size and simulated latency.

PERFORMANCE

AMD EPYC 9374F · RHEL 9 · g++ 15 · -O3 -march=native

The usual objection to actors is messaging overhead. The answer is `fast_send`, which runs the receiver's handler inline on the caller's thread and returns the reply as a value.

PATH	P50	WHAT IT COSTS
<code>fast_send</code> — inline, no thread hop	~10 ns	≈ one polymorphic virtual call
Queued send, one Group thread	~90 ns	queue push/pop + dispatch
Queued send, separate threads	~3,370 ns	cross-core mailbox wakeup, twice
Pooled round trip (2 messages)	~92 ns	vs ~128 ns with plain new
Allocator tail (max observed)	~33 μs	vs ~5.5 ms unpooled — the durable win

MEASURED Ping → pong → reply, one message in flight, from the microbenchmarks in `actors/cpp/perf`. Grouped ÷ ungrouped is 37× here and 18× on an Apple M3 — the absolutes travel less well than the ratio.

EXPECTED End-to-end **tick-to-trade** sits in the ~100 μs range at the median with p99 under a millisecond. That figure is a design target derived from the component measurements, not a colocated production measurement, and is labelled as such wherever it appears.

SHADOW-PPOV EXECUTION

ES futures · 246 sessions of calendar 2025 · 100 contracts bought and 100 sold per window

Most execution algorithms either cross the spread or quote continuously. Shadow-PPOV takes a third path: on observing a

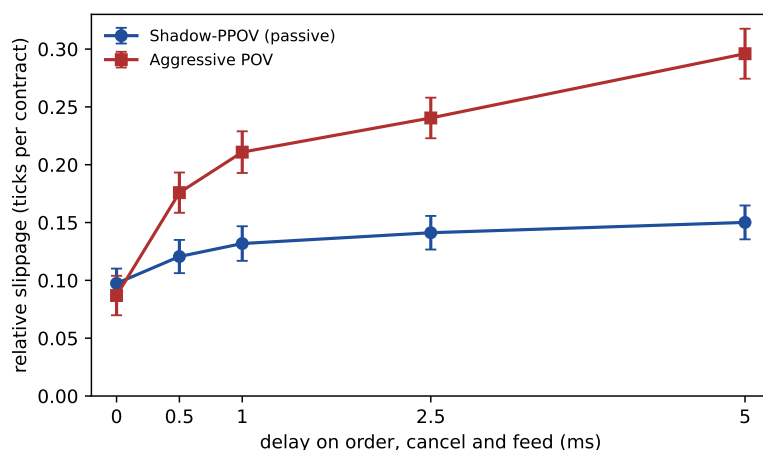
third-party add, it places its own limit order **at that order's price, on that order's venue**, records nothing but the pair of exchange identifiers, and cancels when the order it follows ends. No order-book model. No fill-probability forecast. No routing computation — the venue mix is inherited from the flow rather than solved for.

Each window opens with 100 contracts to buy and 100 to sell, worked at once. **Relative slippage** prices each leg against the mid at the instant the window opened and averages the two, so the market's drift during the window cancels.

ZERO SIMULATED LATENCY	SHADOW-PPOV (PASSIVE)	AGGRESSIVE POV (CROSSING)
Completed windows	9,592	9,581
Relative slippage, ticks per contract	+0.0955 ± 0.0130	+0.0952 ± 0.0135
Aggregate participation	4.46 %	4.86 %
Time to fill 100	72.7 s	62.4 s
Filled on arrival rather than resting	1.04 %	51.86 %

MEASURED Intervals are 95 % and clustered by session. One of these rests for 99 % of its executed quantity and the other crosses for half of it, at matched participation — and the round trip costs the same either way, to the fourth decimal, over more than nine thousand windows each.

What latency costs. The same delay applied to all three paths at once: the outbound order, the outbound cancel, and the inbound market-data feed.



DELAY	PASS.	AGGR.
0	+0.097	+0.087
500 μ s	+0.121	+0.176
1 ms	+0.132	+0.211
2.5 ms	+0.141	+0.240
5 ms	+0.150	+0.296

Passive loses about **0.011** ticks per contract per millisecond of delay and aggressive about **0.042** — roughly four times as fast. The ordering between them reverses inside the first half-millisecond.

The asymmetry is not in what a fill costs. A marketable limit never executes worse than its limit — if the book moves in its favour during the flight it simply fills better. What latency changes is how often it fills at all: when the level it was priced from has been consumed, the order rests at a price the market has already left, and the quantity comes back to be re-sent at whatever the price has become. A resting order has no equivalent failure: a quote that arrives late has still arrived, and pays at most the width it crossed.

WHAT IT DOES NOT CLAIM

- **Adverse selection is not avoided.** The half-spread a resting order captures is returned, in expectation, through adverse selection — which is why passive and crossing cost the same here. Post-fill mark-outs show that cost is complete within a second of the fill and undiminished at thirty.
- **The replay models no market impact.** Orders fill against the recorded feed as though they had not been there. At 4.5 % participation that assumption is mild; at the 6.9 % the passive side reaches under 5 ms of delay it is strained, and the reported participation should be read as an upper bound.
- **Tick-to-trade is a target, not a colocated measurement.**
- **No comparison against a VWAP or TWAP algorithm has been run.** Beating the market's own volume-weighted price over the interval you happened to trade in is a weaker and confounded claim, and the report says so.
- **One instrument, one year.** ES is a one-tick book for most of the session; do not carry the results to wider-spread products without re-measuring.

WRITTEN UP

Shadow-PPOV: Model-Free Passive Execution via Order-Level Shadowing and Identifier-Driven Cancellation — method, prior art, the full measurement corpus, latency sensitivity, and the case for a model-free passive method as the benchmark a predictive placement model should have to beat. **fast_send** — the inline-dispatch design and its measurements. Source, benchmarks and runbooks are in the repository.